

EDUCATION BACKGROUND

Anhui Normal University

Wuhu, China

Bachelor of Engineering in Software Engineering

Sept 2023 – Jul 2027 (Expected)

- Weighted Average Score: 86.7/100
- Relevant Coursework: *Data Structures, Algorithm Design and Analysis, Operating Systems, Computer Networks, Database Principles, Software Engineering, Object-Oriented Programming, Discrete Mathematics, Linear Algebra, Probability and Mathematical Statistics, etc.* 之后按照英文成绩单核对下 然后 雅思成绩我不知道放哪里
- CET-4: 518

[MOVED ↑] AWARDS & HONORS 整段从 ADDITIONAL INFORMATION 之前移到这里 并且重要奖项加粗

AWARDS & HONORS

[删减] 划红线者建议删除。蓝桥杯删除，与 icpc/ccpc 重合，档次更低；节能减排三等奖与计算机方向无关。挑战杯保留奖项本身，划去项目全名。两条科技活动奖学金已合并为一行 2024, 2025

- **Silver Medal, ACM-CCPC 2025 National Collegiate Programming Contest Jinan Regional Contest | Nov 2025**
- **Bronze Medal, 2025 ACPC Asia Wuhan Regional Contest | Oct 2025**
- ~~Second Prize, C/C++ Group B (University), 16th "Blue Bridge Cup" National Finals | Jun 2025~~
- ~~First Prize, C/C++ Group B (University), 15th "Blue Bridge Cup" Anhui Provincial Contest | Apr 2024~~
- ~~Excellence Award, C/C++ Group B (University), 15th "Blue Bridge Cup" National Finals | Jun 2024~~
- First Prize, 22nd "May 1st" Mathematical Modeling Competition | Jun 2025
- Second Prize, National College Student Mathematical Modeling Competition (Anhui Provincial) | Dec 2024
- First Prize, ~~SmartGuard EasyHealth — An AI-Driven Personalized Stroke Rehabilitation Training and Progress Monitoring Platform~~, 11th "Challenge Cup" Extracurricular Academic & Technological Works Competition (University Level) | Mar 2025
- ~~Third Prize, 2nd "Han Gu · Jingyu Cup" Anhui College Students Energy Conservation and Emission Reduction Competition | Apr 2025~~
- ~~First Prize, Anhui Normal University 16th ACM Programming Contest | 2025~~
- Science and Technology Activity Scholarship, Anhui Normal University | 2024, 2025

HIGHLIGHTED TECHNICAL SKILLS

[压缩/新增] HIGHLIGHTED TECHNICAL SKILLS 由 8 组 19 条合并为 4 组 11 条；删去重复项与自我评价式表述；组播行情分发 / DPDK 暂撤

- **C++ Performance Engineering**
 - System and kernel tuning: CPU affinity and core isolation, NUMA node-local allocation, real-time thread priorities, IRQ affinity, huge pages, kernel and BIOS tuning for jitter.
 - Memory, cache and code generation: cache-line alignment and false-sharing avoidance, memory pools and pre-allocation, struct layout, prefetch, branch prediction, inlining, SIMD / AVX intrinsics.
 - Lock-free concurrency: lock-free queues with micro-batching, SPMC shared-memory queues, double-buffer plus atomic index, custom spinlocks, C++20 coroutine scheduling.
 - Measurement and bottleneck analysis: cycle-level latency measurement (RDTSC), p50/p95/p99 latency histograms, the roofline model, top-down microarchitecture analysis (TMA), perf and flame graphs.
- **Systems & Networking**
 - Linux systems programming: process management, IPC, file I/O, epoll multiplexing, mmap; thread pools, producer-consumer, condition variables, connection pools, event-driven and state-machine designs.
 - Protocols: TCP/IP, HTTP, WebSocket, JSON-RPC 2.0; application-layer framing (Content-Length, custom binary protocols); TCP_NODELAY vs. Nagle for small messages.
- **AI & Agent Engineering**
 - MCP and agent infrastructure: implemented a full MCP Server framework in C++17 from scratch (Tool/Resource/Prompt registration, scheduling loops, JSON-RPC dual transport); multi-agent orchestration as a state graph with LangGraph.
 - LLM integration: API integration, prompt engineering, structured-output constraints, multi-turn dialogue orchestration, RAG; local inference with Ollama; end-to-end ASR/TTS/VAD pipelines.
 - Working with AI coding agents: daily use of Claude Code and Cursor for development and debugging; building agent-driven pipelines that carry an engineering task end to end, from generating candidate changes to building, verifying and benchmarking them automatically.
- **Languages & Engineering Quality**
 - C++ (C++17: STL, RAII, smart pointers, template metaprogramming, the Pimpl idiom); Python for tooling and agent orchestration.
 - CMake, vcpkg, Git; GTest unit testing, AddressSanitizer, Cppcheck static analysis, spdlog structured logging.

PROFESSIONAL EXPERIENCE

C++ Engineer (Intern) | Nanjing Pushi Intelligent Technology Co., Ltd., Nanjing, China

Apr 2026 – Present

Intro: High-frequency quantitative trading systems; C++ high-performance programming, Linux systems development and low-latency optimization. Two C++ repositories: the factor-computation engine (function-level performance attribution) and the trading service (TWAP position push, bidding-strategy endpoints, position aggregation and query).

- Reopened a module that stage-level analysis had already written off as near its algorithmic floor: function-level profiling on a network-isolated devbox put ~3.4% of runtime in kernel page-fault and page-zeroing paths, so I ran a paired A/B on an allocator swap via LD_PRELOAD — same binary, no recompilation, sample size fixed in advance. Median wall clock fell 4.77%, all six deltas in the same direction, permutation $p = 0.0312$, factor output byte-identical.
- Met the delivery target on the TWAP position-push path at 5,000 instruments $\times$ 10 users: zero dropped pushes and correctness PASS at both verification tiers (10,000 and 50,000 pushes), p95 held at 1.2 ms. Moved blocking database writes onto an async executor off the hot path, gave each session a dedicated push queue, removed hot-path logging, and drained Redis in batches.
- Brought five bidding-strategy endpoints (add / edit / start / stop / delete) into the millisecond range under the same load configuration, max p95 45.7 ms, by making a full-list Redis refresh asynchronous and coalescing requests in a short window.
- Ran the work as a recorded change ledger rather than a set of one-off wins: 5 changes landed, 4 reverted, each revert with a written reason. Raising the Redis drain batch to 256 passed once at the stress tier but lost pushes on all three repeats, so that tier was recorded as an unsafe operating point rather than claimed.

RESEARCH & PROJECTS

[改名] RESEARCH PROJECTS → RESEARCH & PROJECTS: 本节除独立研究外还包含 MCP 框架、语音平台、暑期社会实践（微信小程序），原名与内容不符

Independent Research | Agent-Driven C++ Performance Optimization and Speedup Verification

Jul 2026 – Present

Link: github.com/Jasper-aa64/promotion-gate-audit

Intro: Built an agent-driven C++ performance-optimization loop: an AI agent proposes and instantiates candidate patches while the harness automates build, correctness checking, paired benchmarking, and evidence archiving. Candidate generation is separated from merge authority, so "can produce a patch" never implies "the patch should be accepted."

- Defined what counts as a real speedup — minimum practical effect, paired measurement, bootstrap uncertainty, directional consistency, staged sample budget — and ran a preregistered evaluation on the public C++ library fmt: 0 of 30 null patches promoted, against 16 under a naive first-observation rule on the same data. All 30 patches were no-ops that compiled to binaries byte-identical to the control, so this run only tests whether the pipeline reports a speedup that isn't there; it says nothing yet about how the rule behaves on patches that really change the code.
- Audited the criteria themselves and showed the directional-consistency rule was unreachable as originally specified: certifying it at 95% confidence needs 29 paired runs all pointing the same way (under an IID assumption), against a sample budget of 4/8/12. Revised the experiment contract accordingly. Data, scripts, and the technical report (v0.1) are public.

Main Developer | Development of C++ MCP Server Framework & Multi-Agent Coding System

Mar 2026 – Apr 2026

Intro: Implemented the MCP protocol in C++17: pluggable handler dispatch for Tools, Resources and Prompts with dynamic registration and runtime routing, over a JSON-RPC dual transport layer (HTTP + SSE / stdio + Content-Length framing) behind abstract interfaces.

- Extended the server with 13 C++-specific tools (file I/O, code execution, complexity analysis, Git) behind a three-layer security mechanism: path sandbox, command whitelist, and a quote-aware dangerous-symbol filtering state machine.
- Applied fine-grained concurrency control with independent mutexes per module to avoid cross-module lock contention, and graceful shutdown via std::atomic and signal handling so in-flight requests complete and every resource is released on exit.
- Orchestrated three agents (Coder, Reviewer, Tester) as a LangGraph state graph with up to three VERDICT-driven retries, role-based tool permissions, and both hosted LLM API and local Ollama backends; collected thought/action/observation trajectories and exported them as SFT and DPO training data.

Main Developer | AI Real-Time Voice Platform (Interview & IELTS)

Feb 2026 – Aug 2026

Link: github.com/Jasper-aa64/AI-realtime-speech-for-IELTS-Job-Interview

Intro: Independently designed and built a C++17 real-time voice platform integrating ASR, TTS, VAD and LLM, then ran two separate applications on the same core: an AI technical-interview system and an IELTS speaking and writing trainer.

- Structured the platform in four layers (Common, Service, Business, UI) decoupled by abstract interfaces, with a thread-safe global state machine as the sole coordination center across 8 transitions, and a 4-thread model (main, microphone, playback, receiving) coordinated by std::atomic, std::mutex and std::condition_variable.
- Built the audio and transport path: cross-platform capture and playback at 16 kHz int16 in and 24 kHz float32 out, a 200 ms blocking frame balancing latency against CPU, and a Boost.Beast WSS persistent connection carrying a custom binary protocol (header / event / compression).
- Drove the LLM through an OpenAI-compatible libcurl client constrained to structured JSON for resume-driven question generation, real-time scoring and follow-up decisions; isolated Boost.Beast, PortAudio and PoDoFo behind

Pimpl with no dependency leakage in headers, with spdlog, GTest and AddressSanitizer, and a Qt6 desktop GUI exporting JSON reports.

- Built the IELTS trainer on that core around an interactive report: every sentence of the user's output carries its own evaluation, the report is editable in place, accepted phrasings are promoted into a personal corpus, and the revised text is resubmitted for re-scoring.

Team Leader | **Summer Social Practice (App Development Program)**

Jul 2025 – Aug 2025

- Led the development of a full-stack WeChat mini-program for exhibition introduction and indoor navigation.
- Managed task division, progress control, and official documentation including requirements and API specs.
- Successfully delivered and launched the program; obtained the National Software Copyright Certificate.

[MOVED ↓] 原 AWARDS & HONORS 整段已上移到 EDUCATION BACKGROUND 之后

ADDITIONAL INFORMATION

- Languages: Chinese (Native), English (Proficient in academic & technical communication)
- Skills: Erhu (National Conservatory of Music Level 10), Chinese Painting